

ScamDunk: What I Fixed, and What's Left

Plain-language summary for the owner · 12 June 2026 · All work is saved on the `claude/compassionate-clarke-s6xqk1` branch in GitHub

The short version

I reviewed your entire app, found 272 issues, and **fixed essentially all of them in code** — 17 batches of changes, all tested (the app compiles, all automated tests pass, and a real production build succeeds). The work is safely in GitHub but **not yet live**.

To make it live and finish the last few items, I need you to do **5 short things** (about 30 minutes total — see the separate "Your Steps" page). The most urgent: **your database password was visible in the code, so you must change it today**.

A few items are deliberately left for later because they need *your data* or *your decision* — not more coding. Those are listed near the bottom, and you can hand any of them back to me with one click.

The three things you were worried about

1. The AI risk scores weren't trustworthy **FIXED IN CODE**

WHAT WAS WRONG

Two separate problems were making the scores unreliable:

- **Almost every small/penny stock was being flagged "HIGH risk"** — because the system counted the same warning sign 4–5 times over. When everything looks dangerous, people stop trusting the warnings.
- **The "AI brain" wasn't actually running.** One of the two machine-learning models was silently broken and contributing nothing on every single scan; the other was trained on fake practice data, so its answers were noise. What customers saw was really just a checklist, not the AI you thought was running.

- **The version you tested with was different from the version customers used** — so any accuracy you measured didn't reflect reality.

WHAT I CHANGED

- **One scoring system now, used everywhere** — what you test is exactly what customers get.
- Stopped the double-counting, so ordinary stocks are no longer auto-flagged. Missing data is now treated as "unknown" (with a confidence label) instead of silently assuming the worst.
- **Turned the broken AI models OFF by default** so they can't add noise, and fixed the underlying defect so they can be switched back on once they're trained on *real* data (that's one of the "later" items).
- Fixed the regulatory checks (suspended stocks now correctly score HIGH; the fake "SEC list" of placeholder tickers is gone).

2. The social-media scans were unstable and expensive FIXED IN CODE

Money finding: Apify is not used anywhere in your code. I searched everything — the code, the dependency list, even the full history. If you're paying for an Apify subscription, it's money going nowhere. **Cancel it.** Your real scan costs come from three services (Serper, Perplexity, OpenAI), and they were roughly doubled by the instability below.

WHAT WAS WRONG

- **Every full scan was timing out and losing its results** — the scan ran too long for the platform's limit, saved nothing until the very end, and got killed. Then a backup process re-ran the *same* paid searches, paying twice.
- One scanner mathematically *could not* finish in the time allowed — it had never once completed a full run.

WHAT I CHANGED

- **Results are now saved as they come in**, so a timeout no longer wipes out (and re-bills) the whole scan.
- Trimmed the searches (fewer, smarter queries; only pay for the expensive service on stocks that already look suspicious; skip posts already seen). **Estimated 60–80% lower cost per scan** — on top of cancelling Apify.

- Removed a broken, unused "browser agent" feature that was quietly collecting your personal Discord/Twitter/Instagram passwords (a ban-and-legal risk) while doing nothing.

3. Security holes FIXED IN CODE 1 STEP NEEDS YOU

THE URGENT ONE

Your live database password was written directly in the code — anyone who could see the code could open your database and read every customer's data. I removed it from the code, but the password itself is still valid and is in the project's history. **You must change it in Supabase today** (Step 1 on your Steps page).

OTHER HOLES I CLOSED IN CODE

- **Anyone could get unlimited free scans** through a side-door that checked your plan limit but never counted the scan. Fixed.
- **One Apple purchase could unlock unlimited Pro accounts** (the receipt could be reused on any account). Fixed — each purchase is now tied to one account with a real expiry.
- **The login page had no limit on password guessing.** Added rate-limiting.
- **A "preview login" created an admin account with a password printed in the code.** Closed.
- **Deleting your account didn't actually cancel your PayPal billing** — you'd keep being charged. Fixed.
- Added standard web security headers (clickjacking/HTTPS protections) that were entirely missing.

Other important fixes you didn't ask about (but needed)

- **Your deploy process could silently delete live customer data.** Every deployment was running a command that force-syncs the database and can drop columns. I switched it to the safe, industry-standard "migrations" approach with a full rollback trail. FIXED you run it once

- **There was no automated safety net.** Nothing checked the code before it went live — so mistakes were first discovered in production (where they could damage the database). I added an automated check that runs on every change. **FIXED**
- **The admin dashboard could go blank under light load** (it read data from GitHub with a tiny hourly limit; two browser tabs could exhaust it). Reduced the load and made failures visible instead of showing empty screens. **FIXED** (a deeper rebuild is a "later" item)
- **The "upload a screenshot" feature on your scan page did nothing** — images were collected and thrown away, while the screen told users they were being analyzed. On a scam-protection product, that's a trust problem. I removed the misleading feature until the backend can actually process images. **FIXED**
- Dozens of admin-dashboard bugs (pages freezing on a glitch, a draft reply leaking between support tickets, an editor security hole), plus website/SEO fixes. **FIXED**

What's left — and why I didn't just do it

None of these are blocked by more coding. They're blocked by needing your **data**, your **decision**, or your **access**. You can hand any one back to me with a click (next section).

Item	What it means	Why it's waiting
Dial in the score thresholds	Decide the exact line between "Medium" and "High" risk, measured against real outcomes.	Needs your labeled history (which past stocks really were scams). The engine is ready; I just need the data to tune it.
Re-train & re-enable the AI models	Train the two ML models on real scam data, then switch them back on.	Needs a real training dataset. They're safely OFF until then so they can't mislead.
Move the admin data to your database	Stop reading dashboards from GitHub; read from Postgres instead (faster, no limits).	A larger rebuild. I added a stop-gap; the full move is best done with database access + testing.
Upgrade the web framework (Next.js 16)	Clears the last 5 dependency security warnings.	It's a "major version" change that can break things; needs careful testing, ideally after the above is stable.

Final polish	A handful of lower-traffic admin pages and accessibility touch-ups.	Low impact; I prioritised the engine, security, and stability first. Quick to finish on request.
---------------------	---	--

Ask me anything / send work back to me

There's no live chat inside this page. Instead, these buttons **copy a ready-made instruction to your clipboard** — just paste it into the ScamDunk Claude Code session (the same place you've been talking to me) and send. Click one:

Or ask your own question. Type it here, click Copy, and paste it into Claude Code:

Prefer email? You can also forward any of these to your developer, or reply in the Claude Code session. Everything I've done is already in GitHub, so any teammate can review it there.

Companion documents: **Your Steps** (the 5 things to do) · the full technical audit (`ScamDunk-Code-Audit-2026-06-11.pdf`) · the implementation status (`IMPLEMENTED.md`). All in the `audit/2026-06-11/` folder of your repository.